Document Processing System (Java)

Problem

The system receives events representing documents that need to be processed. Each event contains a reference to a file already stored in a local storage system and some contextual metadata. The goal is to implement a backend component that retrieves the document, processes it, and produces a structured output.

Example input event with type **DocumentReceived**:

```
{
  "documentId": "123",
  "storageRef": "input/invoice.pdf",
  "metadata": {
    "type": "invoice|credit_note",
    "receivedAt": "2026-05-01T10:00:00Z"
  }
}
```

The storageRef field represents a file stored on the local filesystem.

Execution

The system must process each event by performing the following steps.

First, the event must be validated. In particular:

- **documentId** must be present and not empty
- **storageRef** must be present and point to a valid file
- **metadata** must be well-formed

After validation, the system must:

- read the file from storageRef
- load its content as a byte array
- compute a SHA-256 hash of the content
- generate a ZIP archive containing:
- invoice.pdf (original file content)
- metadata.json (must include both the original metadata received in the input event and additional metadata generated during processing)
- hash.txt (computed hash)

The ZIP file must be stored in a local output directory like: output/123.zip

The system must produce (or log) a final event with type **DocumentProcessed**:

```
{
  "documentId": "123",
  "zipPath": "output/123.zip",
  "metadata": {
    "type": "invoice",
    "processedAt": "2026-05-06T12:30:00Z",
    "hash": "abc123...",
    "sizeBytes": 20480
  }
}
```

Instructions

The solution must be implemented in Java. Frameworks usage is optional.

The code should be:

- modular and well-structured
- clearly separated between event handling, processing logic, and storage access
- robust in error handling
- covered by automated tests